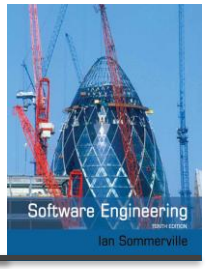
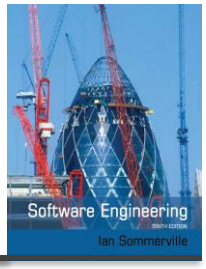
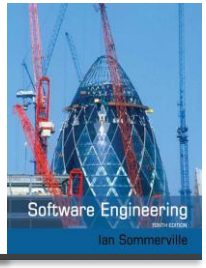


Chapter 8 – Software Testing



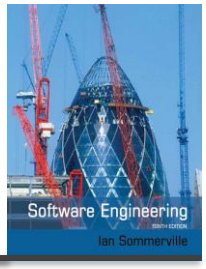
T3 Project!

Topics covered



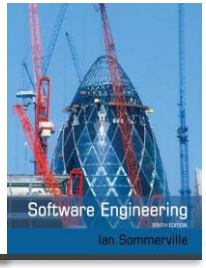
- ✧ Development testing
- ✧ Test-driven development
- ✧ Release testing
- ✧ User testing

Revision!



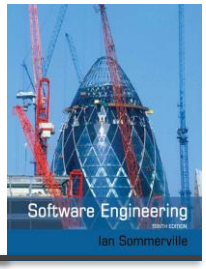
- What is the international standards that guide the organizations that design, develop and maintain products, including software?

Revision!



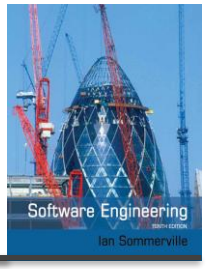
✧ What should Quality managers aim to develop?

Revision!



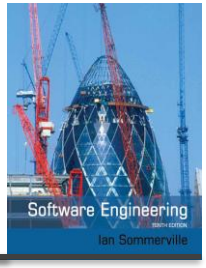
✧ What are the Software quality attributes?

Revision!



- Discuss this 'fitness for purpose' rather than specification conformance.

Catastrophic Effects of Poor Software Quality!



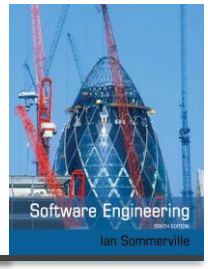
✧ Loss of Human Lives:

- **Therac-25 Radiation Machine (1985–87):** A software bug caused radiation overdoses, killing 3 patients and injuring others.
- **Boeing 737 MAX Crashes (2018–19):** Faulty flight control software (MCAS) contributed to two plane crashes, killing 346 people.

✧ Massive Financial Losses:

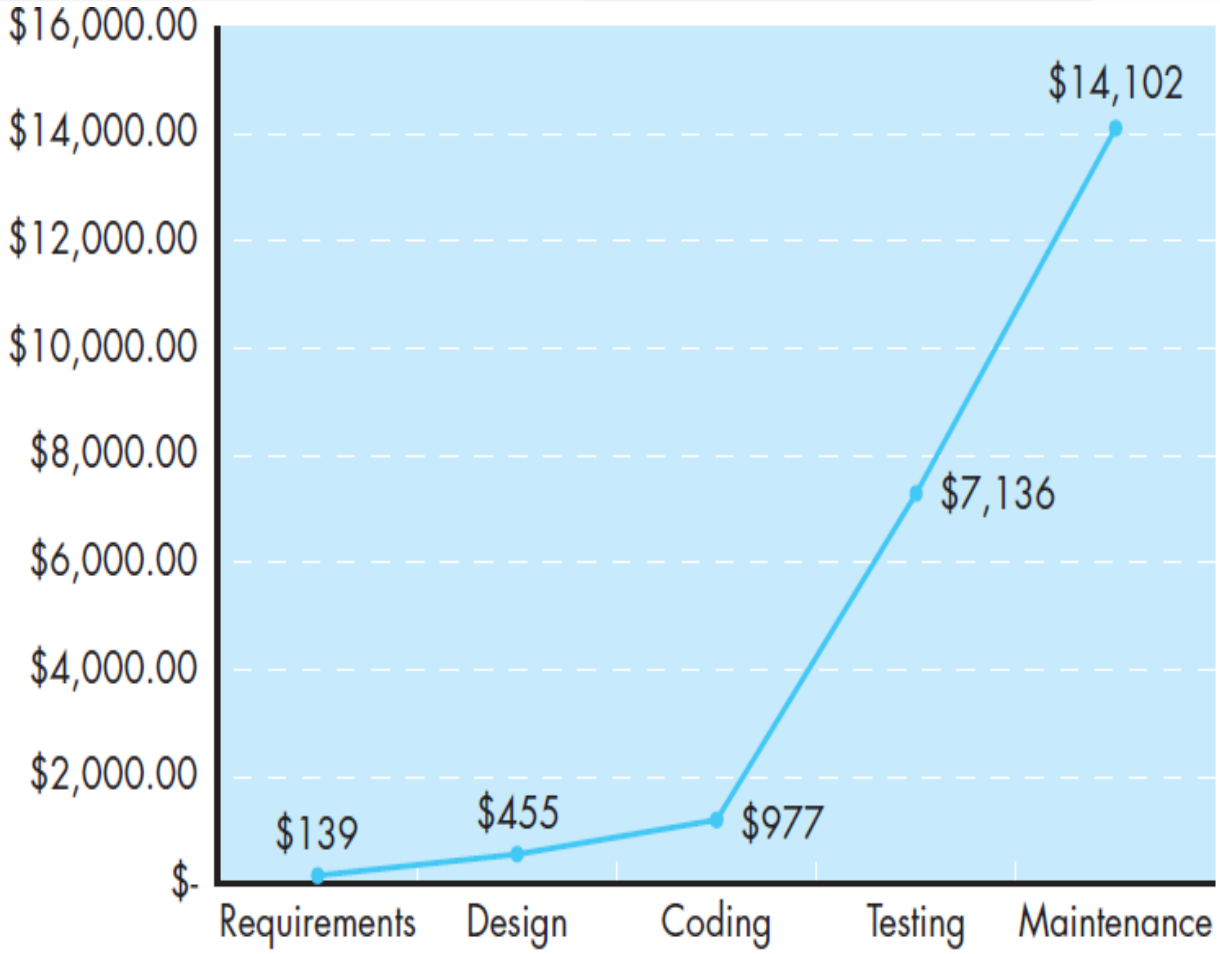
- **Knight Capital Group (2012):** A software glitch in a trading algorithm caused $440 million in losses in 45 minutes, nearly bankrupting the firm.
- **Ariane 5 Rocket Explosion (1996):** A software conversion error caused the European rocket to self-destruct shortly after launch, resulting in a $370 million loss.

Relative cost of correcting errors and defects!

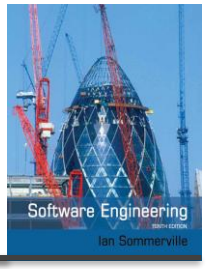


Relative cost of correcting errors and defects

Source: Adapted from [Boe01b].

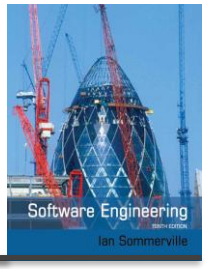


Program testing



- **Testing** is intended to show that
  - a program does what it is intended to do and
  - to discover program defects before it is put into use.
- **When you test software**,
  - you execute a program using artificial data.
- You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.
- **Testing** is part of a more general verification and validation process
  - also includes static validation techniques.

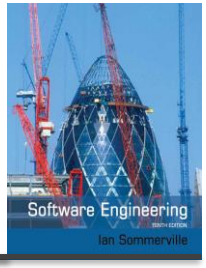
Program testing goals



- To demonstrate to the developer and the customer that the software meets its requirements.
  - **For custom software**, this means that there should be at least one test for every requirement in the requirements document.
  - **For generic software products**, it means that there should be tests for all the system features, plus combinations of these features, that will be incorporated in the product release.

  - **Defect testing** is concerned with rooting out undesirable system behavior. such as:
    - system crashes,
    - unwanted interactions with other systems,
    - incorrect computations and data corruption.

Testing process goals



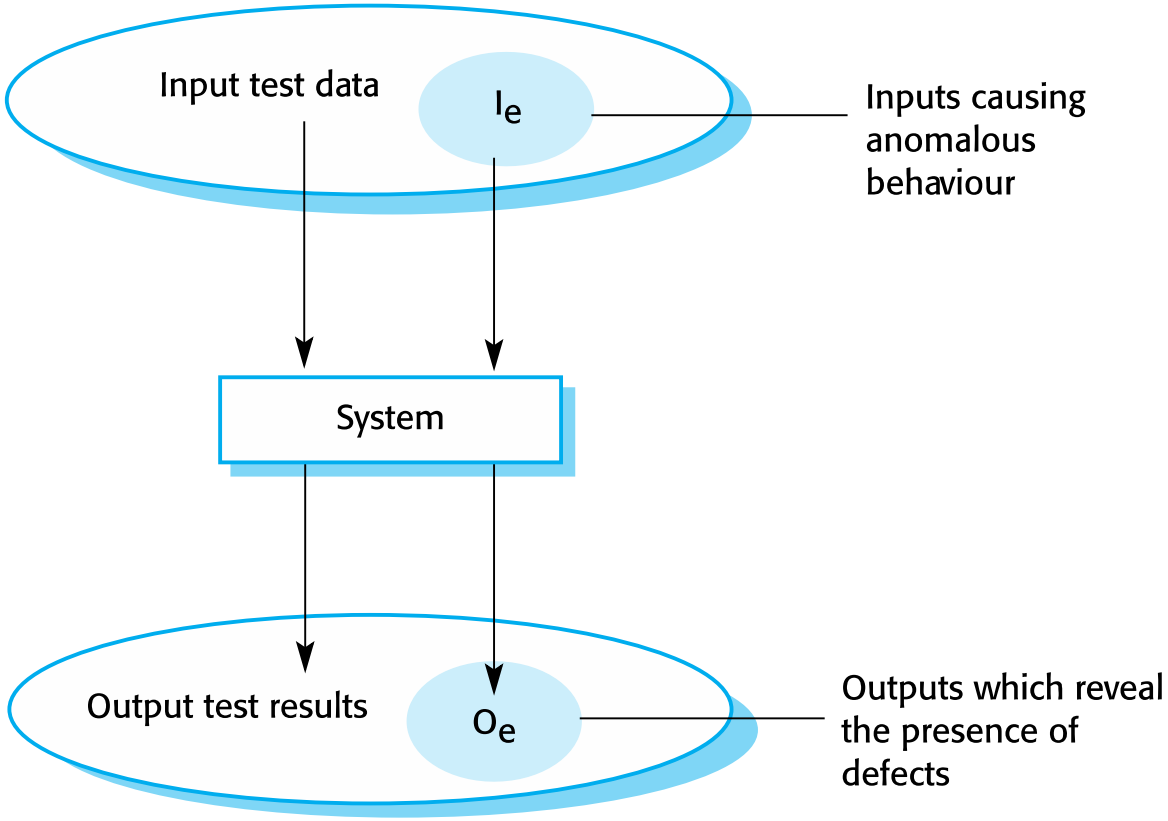
✧ Validation testing - First goal

- To demonstrate to the developer and the system customer that the software meets its requirements.
- You expect the system to perform correctly using a given set of test cases are designed to reflect the system's expected use.
- A successful test shows that the system operates as intended.

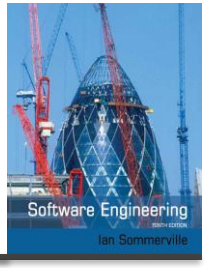
✧ Defect testing - Second goal

- To discover faults or defects in the software where its behaviour is incorrect or not in conformance with its specification
- The test cases are designed to expose defects.
- A successful test is a test that exposes a defect in the system.

An input-output model of program testing



Verification vs validation



✧ **Verification**:

"*Are we building the product right*".

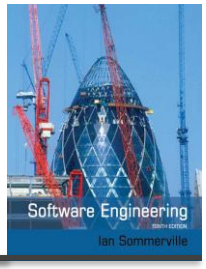
- The software should conform to its specification.

✧ **Validation**:

"*Are we building the right product*".

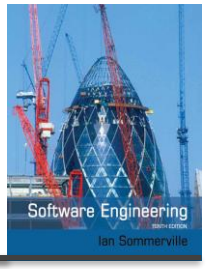
- The software should do what the user really requires.

V & V confidence



- Aim of **V & V** is to establish confidence that the system is 'fit for purpose'.
- Depends on system's purpose, user expectations and marketing environment
  - **Software purpose**
    - The level of confidence depends on how critical the software is to an organisation.
  - **User expectations**
    - Users may have low expectations of certain kinds of software.
  - **Marketing environment**
    - Getting a product to market early may be more important than finding defects in the program.

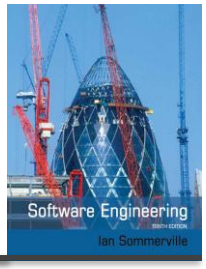
Inspections and Testing



- **Software inspections -** Concerned with analysis of the static system representation to discover problems
  - Static verification Technique - analyze code without running it

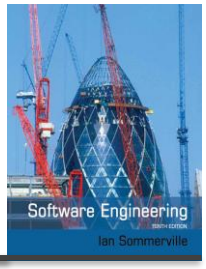
- **Software testing** - Concerned with exercising and observing product behaviour
  - Dynamic verification Technique - executing software under various conditions to identify errors, bugs, and performance issues.
  - Focuses on the behavior of the system during execution.

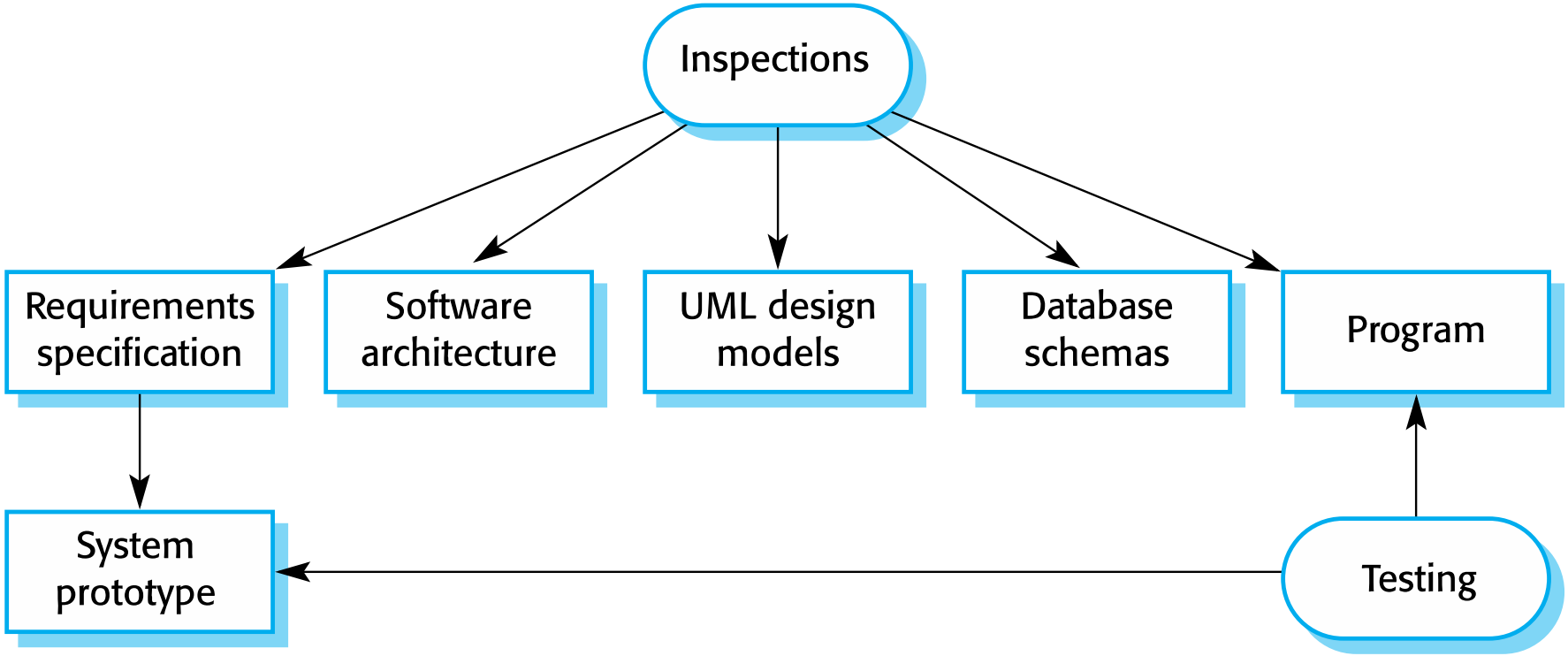
Software inspections



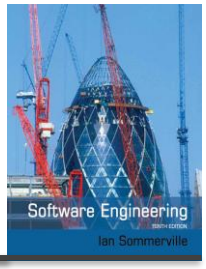
- These involve people examining the source representation with the aim of discovering anomalies and defects.
- They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
- They have been shown to be an effective technique for discovering program errors.

Inspections and testing



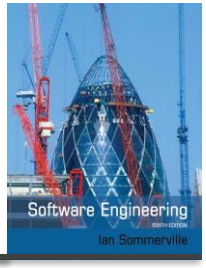


Advantages of inspections



- During testing, errors can mask (hide) other errors.
- Because inspection is a static process, you don't have to be concerned with interactions between errors.
- An inspection can be considered a broader quality attributes of a program
  - compliance with standards, portability and maintainability.

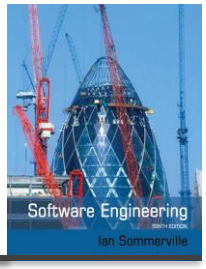
Inspections and testing



- Inspections and testing are complementary and not opposing verification techniques.

- Both should be used during the V & V process.

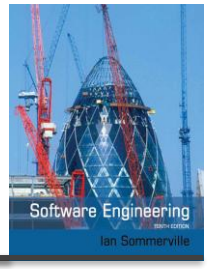
Inspection's Limitation

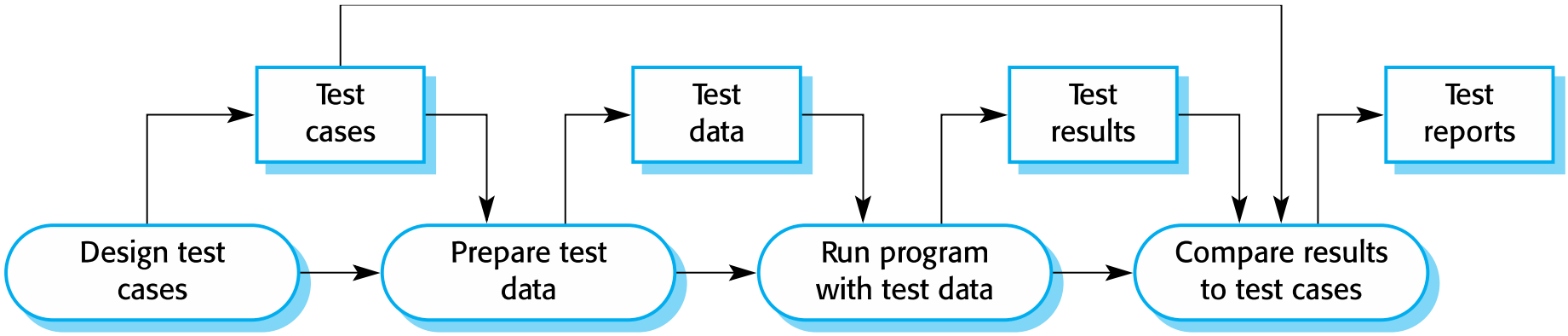


- Inspections can check conformance with a specification but not conformance with the customer's real requirements.

- Inspections cannot check non-functional characteristics
  - such as performance, usability, etc.

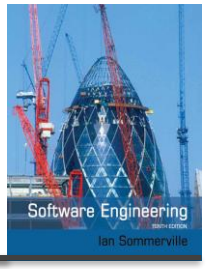
A model of the software testing process



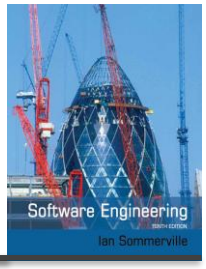


The **test cases** should show that, when used as expected, the component that you are testing does what it is supposed to do.

Stages of testing

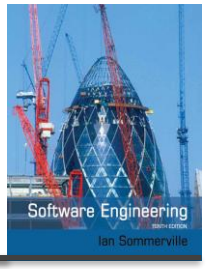


- **Development testing** - system is tested during development to discover bugs and defects.
- **Release testing** - a separate testing team test a complete version of the system before it is released to users.
- **User testing** - users or potential users of a system test the system in their own environment.



Development testing

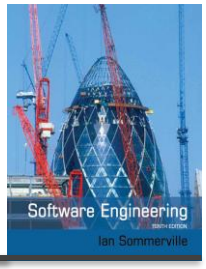
Development testing



Development testing includes all testing activities that are carried out by the team developing the system.

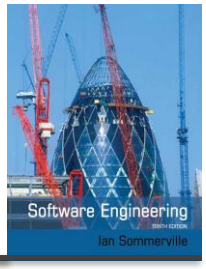
i. **System testing**
ii. **Component testing**
iii. **Unit testing**

System testing



- **System testing** - some or all of the components in a system are integrated and the system is tested as a whole.
  - The focus in system testing is testing the interactions between components.
  - System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces.

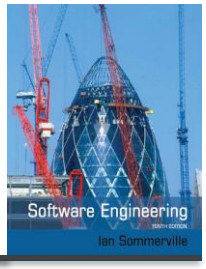
Component testing



- **Component testing** - several individual units are integrated to create composite components.

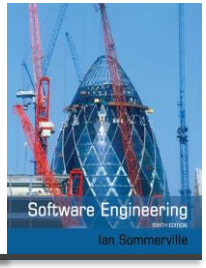
- The focus on component testing to test the interactions between units.

Unit testing



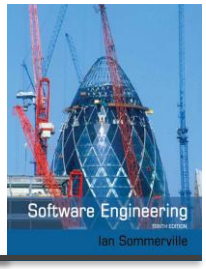
- **Unit testing -** process of testing individual components in isolation.
  - focus on testing the functionality of objects or methods.
    - Can be **automated -** tests are run and checked without manual intervention.

Detecting defects in the component



- **How do you detect defects in the component?**
- This leads to 2 types of unit test case:
  - The first of these should reflect normal operation of a program and should show that the component works as expected.
  - The other kind of test case should be based on testing experience of where common problems arise.
    - It should use abnormal inputs to check that these are properly processed and do not crash the component.

Testing strategies



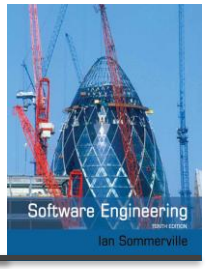
i. **Partition testing**

ii. **Guideline-based testing**

✧ **Partition testing** - identify groups of inputs that have common characteristics and should be processed in the same way.

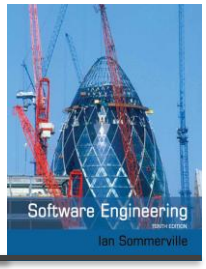
- Choose tests from within each of these groups.

Testing guidelines (sequences)

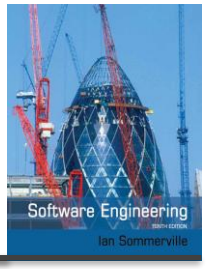


- **Guideline-based testing** - use testing guidelines to choose test cases.
  - These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.
  - Use sequences of different sizes in different tests.
  - Derive tests so that the first, middle and last elements of the sequence are accessed.
  - Test with sequences of zero length.

General testing guidelines

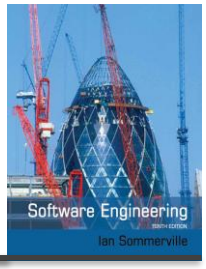


- Choose inputs that force the system to generate all error messages
- Design inputs that cause input buffers to overflow
- Repeat the same input or series of inputs numerous times
- Force invalid outputs to be generated
- Force computation results to be too large or too small.



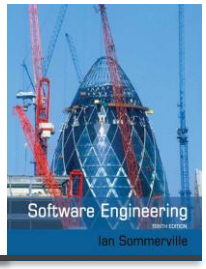
Test-driven development

Test-driven development



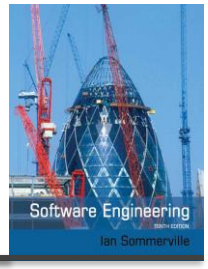
- **Test-driven development (TDD)** approach
  - inter-leave testing and code development.
- **Tests are written before code**
  - 'passing' the tests is the critical driver of development.
- **Develop code incrementally, along with a test**
  - You don't move on to the next increment until the code that you have developed passes its test.
- **TDD was introduced as part of agile methods**
  - such as Extreme Programming.
  - It can also be used in plan-driven development processes.

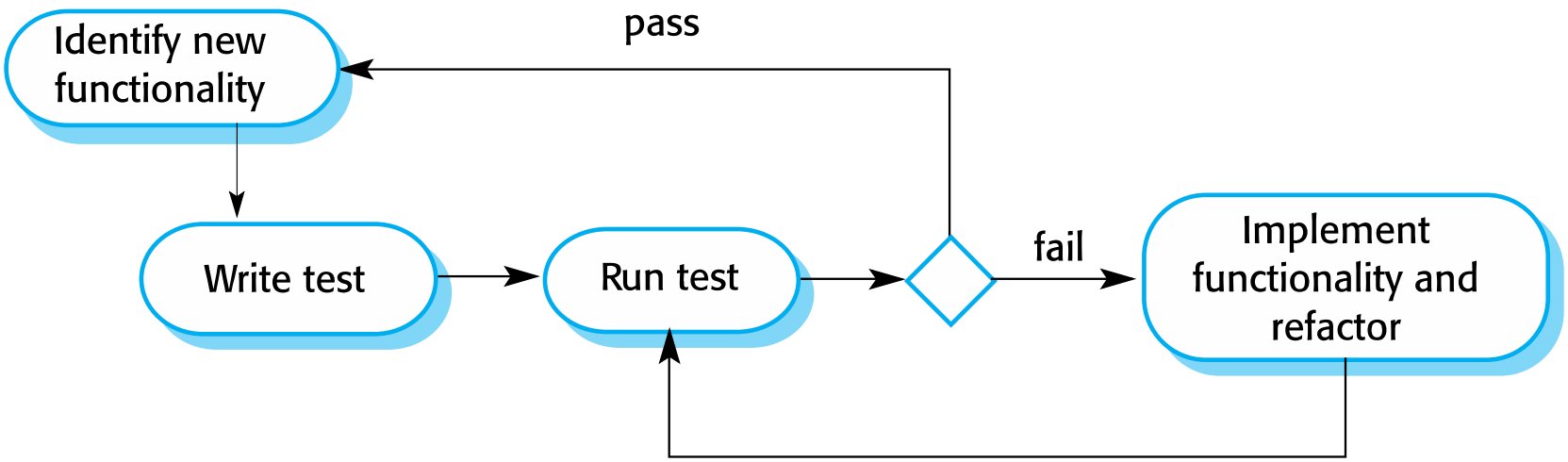
TDD process activities



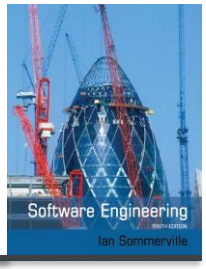
- Start by identifying the increment of functionality that is required.
  - Normally be small & implementable in a few lines of code.
- Write a test for this functionality and implement this as an automated test.
- Run the test, along with all other tests that have been implemented.
  - Initially, you have not implemented the functionality so the new test will fail.
- Implement the functionality and re-run the test.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.

Test-driven development





Benefits of test-driven development



✧ Code coverage

- Every code segment that you write has at least one associated test, so all code written has at least one test.

✧ Regression testing

- All tests are rerun every time a change is made to the program
- ensures that recent code changes have not adversely affected existing functionalities

✧ Simplified debugging

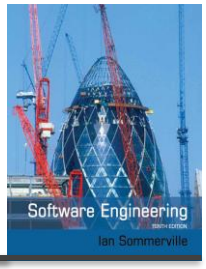
- When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.

✧ System documentation

- The tests themselves are a form of documentation that describe what the code should be doing.

Release testing

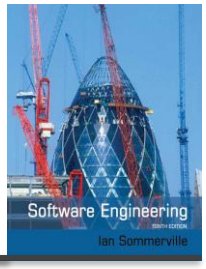
Release testing



- **Release testing** is the process of testing a particular release of a system that is intended for use outside of the development team.
- The **primary goal** of the release testing process - convince the supplier of the system that it is "**good enough**" for use.
  - Release testing, therefore, has to show that the system delivers its **specified functionality**, **performance** and **dependability**, and that it does **not fail during normal use**.
- Release testing is usually a **black-box testing process**
  - Tests are only derived from the system specification.

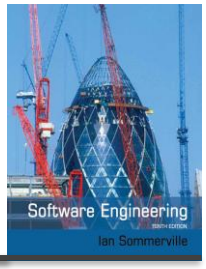
User testing

User testing



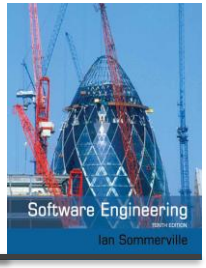
- User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- User testing is essential, even when comprehensive system and release testing have been carried out.
  - The reason for this is that influences from the user's working environment have a **major effect** on:
  - the reliability,
  - performance,
  - usability and
  - robustness of a system.

Types of user testing



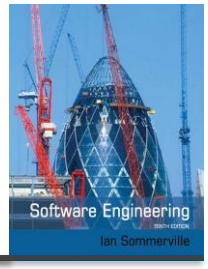
- **Alpha testing**
  - Users of the software work with the development team to test the software at the developer's site.
- **Beta testing**
  - A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- **Acceptance testing**
  - Customers test a system to decide whether or not it is ready to be accepted.

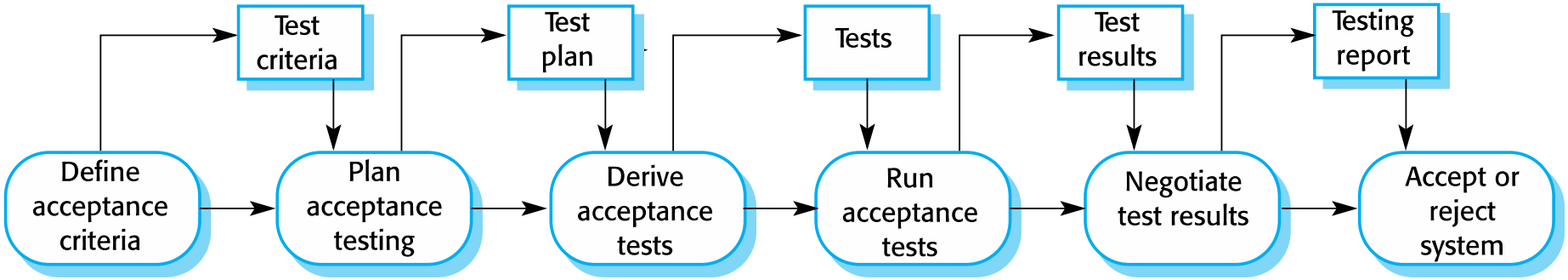
Stages in the acceptance testing process



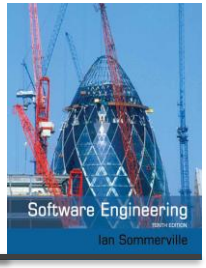
- ✧ Define acceptance criteria
- ✧ Plan acceptance testing
- ✧ Derive acceptance tests
- ✧ Run acceptance tests
- ✧ Negotiate test results
- ✧ Reject/accept system

The acceptance testing process



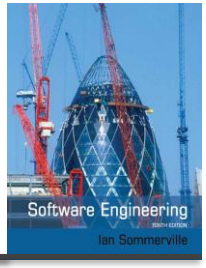


LTD-01!



- How does release testing differ from system testing?
- Discuss requirements-based testing and how it differs from performance testing.

LTD:

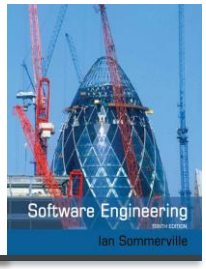


- ✧ Read about Agile methods and acceptance testing.

- ✧ Testing reveals defects but cannot prove their absence. How can software teams ensure reliability beyond traditional testing?

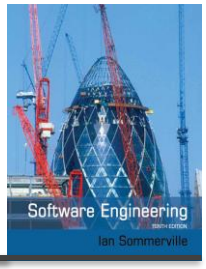
- ✧

Agile methods and acceptance testing



- In **agile methods**, the user/customer is part of the development team and is responsible for making decisions on the acceptability of the system.
- Tests are defined by the user/customer and are integrated with other tests in that they are run automatically when changes are made.
- There is no separate acceptance testing process.
- **Main problem** here is whether or not the embedded user is 'typical' and can represent the interests of all system stakeholders.

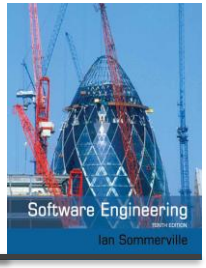
Key points



✧ Testing and Stakeholders

- **Development testing** is the responsibility of the **software development team / Developers.**
- Acceptance testing is **End user** testing process where the aim is to decide if the software is good enough to be deployed and used in its operational environment.
- A **separate team (QA Analysts/Testers)** should be responsible for testing a system before it is released to customers.
  - Design and execute manual & automated test cases. Perform functional, integration, system, and regression testing.
- **Performance Engineers (Performance & Load Testing)** –
  - Test scalability, speed, and stability over/under load.
- Project Managers / Scrum Masters –
  - Ensure testing is aligned with project timelines.

Key points

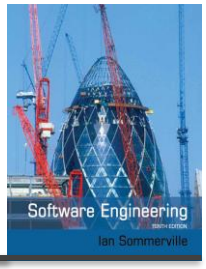


- When testing software, you should try to ‘break’ the software by using experience and guidelines
  - choose types of test case that have been effective in discovering defects
- Wherever possible, you should write automated tests.
  - embedded in a program that can be run every time a change is made to a system.

- **Limitations of Testing: Dijkstra’s principle ("Testing shows the presence, not the absence of bugs") impact quality assurance!**

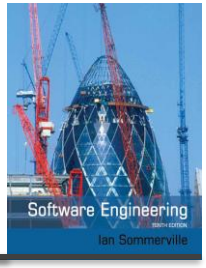
- **LTD: Testing reveals defects but cannot prove their absence. How can software teams ensure reliability beyond traditional testing?**

Scenario-Based Question – Software Testing Stakeholders



- You are the lead QA Analyst in a software development project nearing completion. The development team has completed their coding and conducted internal development testing. The Project Manager and Scrum Master are under pressure to meet a tight deadline and are requesting an early release.
- However:
  - Your QA team has not yet completed the planned system and regression testing.
  - End users have not performed acceptance testing.
  - Performance Engineers have raised concerns about occasional instability during load testing.

Question 1: Multiple Choice



- Which of the following stakeholder groups is primarily responsible for validating the software from a user and business perspective before release?

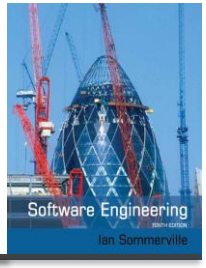
A. Developers

B. QA Analysts

C. End Users

D. Performance Engineers

Question 2: True or False

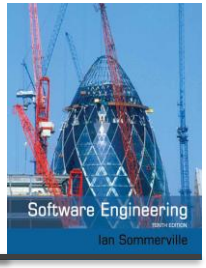


- It is acceptable to skip regression testing if development testing has passed and the release date is near.

A. True

B. False

LTD: Question 3: Scenario Discussion



i. What actions would you take to balance the need for quality assurance with the project timeline pressure?

ii. List at least three key steps and explain which stakeholders you would engage as their roles.